

# Quantum Random Number Generation

Kate Foster  
kfoster1@mines.edu  
CSCI 474

Kate Callan  
kate\_callan@mines.edu  
CSCI 574

Amy Cao  
amy\_cao@mines.edu  
CSCI 474

Chloe Millham  
cmmillham@mines.edu  
CSCI 474

James Towle  
jtowle@mines.edu  
CSCI 474

**Abstract—**In this report we discuss the motivation and results of our group experiment in regard to QRNGs as a replacement for PRNGs. The execution of the experiment is discussed with a thorough analysis of the results and observations. Limitation and potential future work is also discussed, shedding a light on what we hope to grow on in future experiments.

**Keywords—**QRNG, PRNG, Random

## I. INTRODUCTION

Random numbers serve as a vital resource in cryptographic applications to enable unpredictability. Quantum random number generators (QRNGs) are a specific instance of true random number generators (TRNGs) designed to use quantum mechanics to leverage unpredictable physical phenomena rather than mathematical formulas for producing random numbers [1].

QRNGs are pivotal to ensure the generation of secure keys is not susceptible to prediction or reproduction. The current standard for random number generation is to use PRNGs, which have well been established as faking randomness. They use a seed that results in the numbers generated being deterministic. This is a problem as someone who knows the seed can recreate our number generation, getting the exact “random” values generated, which is a clear security issue. This is where the strengths of QRNGs come into play, as the generation of cryptographic key pairs requires high-quality entropy, ensuring that the keys cannot be predicted or brute forced. QRNGs provide this high-quality entropy because of the use of quantum phenomena. This strengthens security by eliminating vulnerabilities that can be exploited by traditional randomness generation [3].

However, QRNGs face drawbacks that limit their practical use in the real world. Measuring quantum phenomena requires expensive, dedicated hardware that can be degraded over time. This poses a risk of decreasing efficiency over time as detectors get influenced by radioactivity [2]. Not only is this expensive to implement, QRNGs are often slower than software-based PRNGs. For most everyday applications, the use of standard PRNGs is sufficient, as computational efficiency and lower implementation costs outweigh the need for perfect randomness. However, for applications that are particularly sensitive and require high security, QRNGs are particularly valuable in ensuring the maximum protection [3].

In this paper, we ask whether QRNG values are truly random. Given an open API QRNG, can we evaluate the quality of this random number generator by testing randomness? We assume that developers who use cryptography in their systems would make the conscious choice between QRNG and PRNGs to encrypt data. Not only is there a firm tradeoff between QRNG and PRNG use; a developer would also want to consider the level of security they want to maintain.

The motivation behind this study was the desire to experiment with testing “random” number generation for true randomness, and to make it more secure in general. The goal of this group study is to test and prove if QRNG would be able to provide truly random numbers, and if it would be a better alternative to PRNGs.

## II. KEY IDEA AND APPROACH

### A. Designing the Experiment

We used the QRNG from Australia National University (ANU), which pulls random numbers from quantum fluctuations of the vacuum, a space that is

devoid of matter or photons. An API key allows for 100 requests a month, for a max of 10 random numbers (uint16) per pull. Each member created an account to access the API functions.

We chose ANU’s QRNG for its real-time quantum entropy source. As a free, easily accessible API service, we determined this would be a fast and convenient QRNG to use for our experimentation. This QRNG also provides a couple unsigned integer data types: uint8 and uint16. The ranges of these data types are  $[2^8 - 1]$  and  $[2^{16} - 1]$  respectively. We chose to go with a larger range of numbers to provide for higher statistical power, which is more likely to detect a non-random pattern if one exists.

Coded in a Jupyter notebook (Python3), we used Json and Requests libraries to pull API calls from our chosen QRNG. A pipeline was implemented to automate the process of pulling requests at random intervals. A notable mention is that random intervals were generated by the Random library, which uses a PRNG. While this random interval can be predicted, this does not pose a risk to the true random number generation as the unpredictable quantum fluctuations would yield a random, unique result even if two people pulled from a QRNG at the same time.

For each request pulled, data was collected into two CSVs: one contained the generated numbers, and the other consisted of meta data on the request, timestamp, duration, success, generated numbers count, and random time interval between pulls. While we ran experiments to test whether the generated random numbers themselves were truly random, we also examined the metadata to determine whether entropy levels fluctuated over time or were influenced by human-designed aspects of the request process, such as pull timing or request success rates. In other words, we wanted to assess not only the randomness of the numbers, but also whether the way we collected them introduced any predictable patterns or entropy variations.

#### B. Data Collection and Merging

All five team members created a free account to access the ANU QRNG API [5] and run the collection pipeline. For each request pulled, the results were stored in two CSVs. Then each team member ran the pipeline twice, once in April and once in May, as the API reset at the beginning of the month. Once this collection was complete, the ten CSVs containing the numbers were merged into a single dataset of 9,870 numbers. Each number in the dataset was converted to a zero-padded 16-bit binary and appended to a bitstream totaling 157,920 bits. This stream was used as the input for most of the tests.

#### C. Statistical Testing Methodology

To test the quality of the numbers produced by the ANU QRNG, we adapted and applied a suite of statistical tests drawn from NIST SP 800-22 Rev. 1a, which is a standard framework for validating random and pseudorandom number generators for cryptographic use [7]. Testing for true randomness relies on quantifying entropy to measure the degree of unpredictability in a sequence of numbers [2]. Each NIST test computes a p-value, and if the result is at or above the threshold, it cannot be rejected. We used a threshold of  $p \geq 0.05$  to be more conservative than the 0.01 minimum set by NIST [7].

We implemented and ran ten tests in total for this project. The Frequency (Monobit) test checks for the global proportion of ones and zeroes. The result of this test should be close to 0.5 [7]. This fundamental property of proportionality was an excellent starting point for the tests. The second test is the Block Frequency Test, which applied that property to each non-overlapping 128-bit block. This would detect any local bias in the sequence the global test would have missed.

The Runs test counts the number of unbroken sequences of identical bits and tests if they alternate at an expected rate [7]. Insufficient run counts can indicate clustering and too many runs can indicate rapid oscillation. The Longest Run of Ones Test was designed to look at each 128-bit block and find the longest run. If this matches the theoretical expected distribution the test will pass. Failure of this test would reveal burst behavior that aggregate counts would miss [7].

The Serial test observed and recorded the count of the four possible 2-bit overlapping patterns to see if they occurred at equal rates. This would catch any short-range correlations between adjacent bits [7]. The Approximate Entropy test compares the frequency of overlapping 3-bit and 4-bit patterns to measure the complexity of the sequence. The theoretical max for a perfectly random sequence is  $\ln(2)$  [7].

The Cumulative Sums test walks through the bitstream and, starting from zero, +1 for every one it encounters and -1 for every zero it encounters. This finds the maximum deviation from zero and will check if it is in the expected range [7]. This will run starting from both the front and the back of the bitstream to calculate the deviation in both directions. The Discrete Fourier Transform Spectral test converts the bitstream to a frequency domain to detect periodic features, which would appear as spikes above the 95% threshold [7].

The last two tests we derived are for the raw integers. A  $\chi^2$  test grouped the 9,870 values into 256 equally sized bins across the uint16 range to ensure uniform

distribution. The Kolmogorov-Smirnov test compared an empirical cumulative distribution function of the output against uniform distribution. These two tests are not part of the officially recommended NIST tests, but they are standard checks for integer level uniformity.

### III. ACTIVITIES

#### A. QRNGs compared to PRNGs

PRNGs are not truly random, as they follow a deterministic algorithm, which means repetitions are inevitable (creating security risks). In comparison, QRNGs are able to generate truly unpredictable numbers by basing randomness on quantum processes. In theory, QRNGs always outperform PRNGs, and there are now measurable results to support these claims [6]. With PRNGs, "randomness" was recreated more frequently as more generations were created, while QRNGs kept the sequences random even on a larger scale. QRNGs show a low variance in experiments and remain stable across runs, meaning results were consistently reproducible. Overall, QRNGs are more effective in creating "random" values, with the generation outperforming PRNGs but taking more computing power.

#### B. Strengths and Limitations of QRNGs

QRNGs do a phenomenal job generating random numbers as we have seen. Their ability to create true randomness is very useful for security purposes, as when set up correctly a QRNG generates outcomes that are impossible to predict. This means that when it is used inside cryptographic ciphers, attacks on this cipher have a much harder time, generally increasing the security of the cipher. Unfortunately, due to the fact that most QRNGs are based on information gathered from the real world, they are significantly harder to set up compared to PRNGs, requiring hardware like cameras or microphones as well as the computing power to gather the data. Along with that, QRNGs are slower to generate numbers, as the computations required to generate one number are more costly than PRNGs, making it less useful for instances where lots of numbers need to be generated at very high speeds. Those aren't the only issues with QRNGs though, as sources of true randomness are extremely sensitive, meaning that any background noise or information can worm its way into the data, causing bias that must be factored out which can add to the runtime.

### IV. DESCRIPTION AND ANALYSIS OF RESULTS AND OBSERVATIONS

#### A. Statistical Test Results (NEW)

The ANU QRNG output passed all ten statistical tests. Table 1 summarizes the full results.

TABLE I: STATISTICAL TEST RESULTS

| Test                       | Key Statistic             | P-Value | Result |
|----------------------------|---------------------------|---------|--------|
| Frequency (Monobit)        | $S_{\text{obs}} = 0.6945$ | 0.4874  | Pass   |
| Block Frequency (M=128)    | $\chi^2 \chi^2 = 1211.84$ | 0.6609  | Pass   |
| Runs                       | $V_n V_n = 78,927$        | 0.8690  | Pass   |
| Longest Run of Ones        | $\chi^2 \chi^2 = 4.2985$  | 0.5073  | Pass   |
| Serial (2-bit)             | -                         | 0.7743  | Pass   |
| Approximate Entropy        | ApEn = 0.693135           | 0.8656  | Pass   |
| Cumulative Sums (Forward)  | Max = 445                 | 0.5240  | Pass   |
| Cumulative Sums (Backward) | Max = 358                 | 0.7216  | Pass   |
| DFT (Spectral)             | d = 0.6235                | 0.5330  | Pass   |
| $\chi^2$ Uniformity        | $\chi^2 = 251.16$         | 0.5563  | Pass   |
| Kolmogorov-Smirnov         | D = 0.0099                | 0.2836  | Pass   |

Out of the 157,920 total bits collected, 79,098 were ones and 78,822 were zeroes. That gives us a proportion of 0.5009 ones and the Monobit test returned  $S_{\text{obs}} = 0.6945$  and  $p = 0.4874$ . The result shows the output is globally balanced within 0.09% of a 50/50 split. If the ANU PRNG has a weak seed or biased hardware, this test would most likely fail.

The Block Frequency Test ran across 1,233 128-bit blocks and returned  $p = 0.6609$ . A sequence can appear globally balanced while containing stretches where ones or zeros appear more often. The high p-value means that localized bias was not present across any portion of the bitstream.

The Runs test identified 78,927 runs and returned a p-value of 0.8690. This test result proves that bits are transitioning between zero and one at a natural rate at the time we generated our numbers. A number generator with memory effects or short-range correlations would produce runs of excessive or insufficient length.

The Longest Run of Ones test looked at the observed vs expected length distribution across six bins shows in Table II and Figure I.

TABLE II: LONGEST RUN OF ONES BIN DISTRIBUTION

| Bins | Observed | Expected |
|------|----------|----------|
|------|----------|----------|

|   |        |     |       |
|---|--------|-----|-------|
| 4 | $\leq$ | 158 | 144.8 |
|   | 5      | 290 | 299.6 |
|   | 6      | 323 | 307.4 |
|   | 7      | 199 | 216.0 |
|   | 8      | 132 | 126.6 |
| 9 | $\geq$ | 131 | 138.6 |

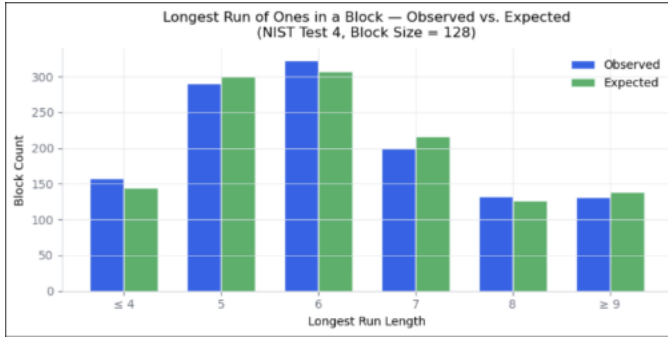


FIGURE I: LONGEST RUN OF ONES

The  $\chi^2$  value 4.2985 and p-value 0.5073 show the variation of streak lengths fall within the expected range. The bin counts only deviate from the expected counts by 13 at most which is negligible across 1,233 blocks of data. A random number generator prone to burst behavior, where extended runs of ones are produced in certain conditions, would show heavy skew in the last bin. Our observed bin size in the last bin was 7.6 smaller than the expected size, so this behavior did not occur at generation time.

The Serial test 2-bit pair counts are shown in Table III.

TABLE III: SERIAL TEST 2-BIT PAIR COUNTS

| Pair  | Observed | Expected |
|-------|----------|----------|
| (0,0) | 39,359   | 39,480   |
| (0,1) | 39,463   | 39,480   |
| (1,0) | 39,463   | 39,480   |
| (1,1) | 39,635   | 39,480   |

All four 2-bit pairs show up in the bitstream at an almost identical rate. The largest deviation from the expected value of 39,480 was 115, which is less than 0.4% off the expected value for the pairs. This proves that short-range correlations between neighboring bits are not present in our bitstream. This specific property is crucial to cryptographic key generation, where adjacent bits must be independent [7].

The Approximate Entropy test returned a value of 0.693135, which is only 0.000012 from the theoretical

$\max \ln(2) = 0.693147$ . This quantifies the predictability of the sequence by measuring the pattern of repetition at the block level. The proximity to the theoretical maximum means the sequence does not contain structures that could be exploited.



FIGURE II: CUMULATIVE SUMS

The tests ran to find the Cumulative Sums passed both forewords and backwards and the maximum excursion for either direction was under 500. That symmetric result is notable because an asymmetric result, where one value drifts drastically more than the other, would indicate drift in the generator during the collection period.

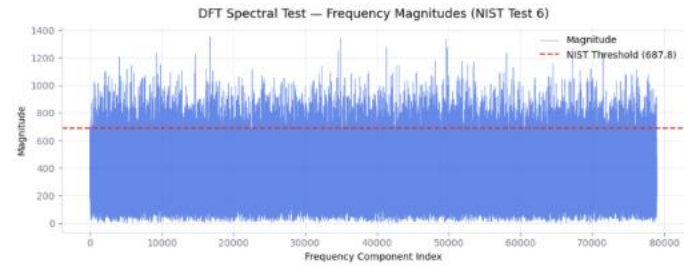


FIGURE III: DISCRETE FOURIER TRANSFORM SPECTRAL TEST

The Discrete Fourier Transform Spectral test found 75,039 frequency components below a threshold of 687.81 against an expected 75,012, resulting in  $d = 0.6235$  and  $p = 0.5330$ . The 27 component difference from the expected falls well within a normal range [7] and no periodic patterns were detected at any point in the frequency domain.

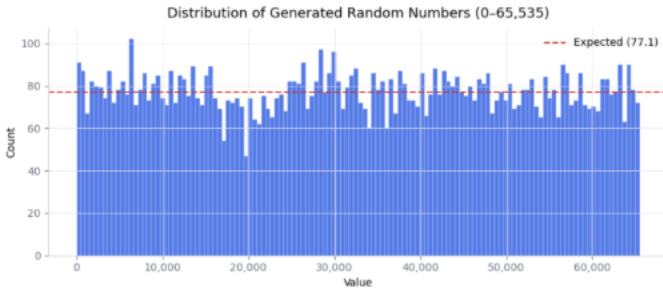


FIGURE IV: DISTRIBUTION OF GENERATED RANDOM NUMBERS

The  $\chi^2$  Uniformity test divided the 9,870 values into 256 bins and found counts ranging in size from 20-57 compared to the expected size of 36.8 per bin. The deviation in bin size is consistent with what random sampling from a uniform distribution produces at this sample size.

The Kolmogorov-Smirnov statistic measures the size of the largest derivation from the empirical to the theoretical cumulative distribution, which was 0.0099 for our data. The output never strayed further than 1% from perfect uniformity at any point. Both integer level tests together confirm the raw integers are uniformly distributed across the full unit16 range from 0 to 65,535.

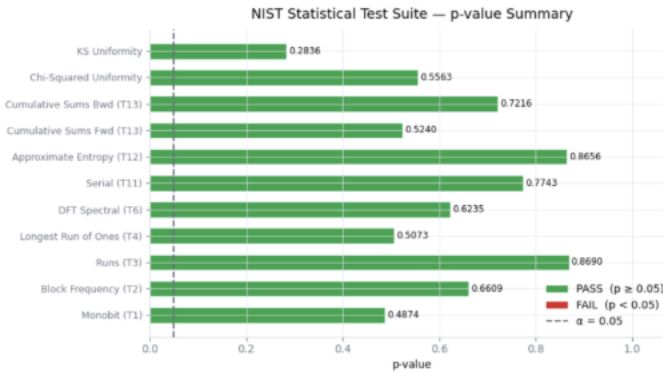


FIGURE V: NIST TEST RESULTS AND P-VALUES

None of the test results yielded a p-value below 0.28. Given the significant threshold of  $p \geq 0.05$ , every result cleared this by at least a factor of five, shown by Fig. V. The output shows no statistically detectable divergence from true randomness across frequency, distribution, correlation, entropy, or spectral dimensions. This consistent results across all ten tests strongly support the conclusion that the ANU QRNG produces truly random unit16 values.

## V. LIMITATIONS AND FUTURE WORK

### A. Sample Size Constraints

Our data was collected using the free tier of the ANU QRNG API [5] which limits each user to 100 requests per month with a limit of ten numbers per request. The five team members all ran the collection pipeline twice, once

in April and once in May. Over those ten collections we collected 9,870 numbers, 157,920 bits, after accounting for initial tests and failed requests. The NIST SP 800-22 standard recommends a minimum of one million bits for running the test suite, while noting that some tests are more sensitive to this length requirement than others [7]. While the tests we were able to run found that our results were consistent with statistical randomness, a larger dataset would strengthen those findings.

### B. Pros and Cons of QRNGs

There is an array of pros and cons to using QRNGs for randomization. QRNGs are currently being used for multiple different things, key generation, scientific and computational simulations, secure communications, and public randomness beacons, for example. The growing use of QRNG in everyday processes has created a visible trend of an increase in reliability and security, as well as improvement in statistical precision and bias minimization on models. This comes from the fact that QRNGs are impossible to predict due to quantum phenomena. This overall improvement in true randomness as well as security truly shows the benefits that come with using QRNGs, and how they could change the future of cybersecurity.

With these pros, however, come cons. Despite QRNGs clearly being more effective in regard to security and randomness, we have to call the efficiency of it all into question as well. Due to how hardware heavy and new the technology and processes are in regard to QRNGs, it takes much longer to create than a PRNG. Accessibility also takes a hit because of this, with the novelty of it all making the costs of the required machinery astronomical. Much more so than PRNGs. As this technology becomes more widespread and then decreases in price, these issues should resolve, but as of now, for most it is simply not realistic to utilize QRNGs for their projects and use.

### C. QRNGs with Hashing

An important question that we discussed in the execution of this research is should QRNGs be used in parallel with hashing. We found the answer to this question to be yes, as hashing adds an extra layer of security that will only help protect sensitive information and increase our security performance. Hashing can also help reduce any bias that might affect our QRNG results and distribution, as well as just make our QRNG results more usable and reliable. Finally, hashing our QRNG results can also help make QRNGs more accessible, reducing runtime and therefore reducing cost and resource allocation. All of these effects clearly show that hashing should absolutely be used in the creation and use of QRNGs.

## VI. CONCLUSION

We set out to determine whether the ANU QRNG produces truly random numbers and if it would be a good alternative to PRNGs in cryptographic settings. After we ran tests on the data, we found that the answer to both of those questions was yes. All ten of our tests passed and the p-values fell far above what NIST recommends. The frequency, distribution, correlation, entropy, and spectral dimensions all proved to be indistinguishable from true randomness.

QRNGs and PRNGs both have their places, and we are witnessing their applications actively shift. For most everyday applications that require a random number generator, PRNGs are the cheaper, quicker, and easiest option. However, when a situation demands true unpredictability, like many cryptographic applications, PRNGs are obsolete compared to QRNGs due to their deterministic nature. Our test results reflect this.

The main limitation of this study was the sample size. With only 157,920 bits, we were well below the one million bit minimum required for NIST SP 800-22, meaning some tests from the testing battery were cut and others had less statistical power. A future study on a larger dataset would strengthen our findings and allow for all fifteen tests to be run. Comparing multiple QRNG sources against a known PRNG would also add useful context to the conclusions we were able to draw.

Quantum hardware will become more accessible in the near future as the cost of entry declines. The case for adopting QRNGs in security-critical systems will continue to grow. This study is a small but concrete step toward understanding what that adoption looks like in practice, and what it takes to verify that a QRNG produces true randomness.

## REFERENCES

1. Wikipedia contributors. (2025, December 9). Hardware random number generator. Wikipedia. [https://en.wikipedia.org/wiki/Hardware\\_random\\_number\\_generator#:~:text=Hardware%20random%20number%20generator%20%2D%20Wikipedia](https://en.wikipedia.org/wiki/Hardware_random_number_generator#:~:text=Hardware%20random%20number%20generator%20%2D%20Wikipedia)
2. A comprehensive review of quantum random Number Generators: concepts, classification and the origin of randomness. (n.d.). <https://arxiv.org/html/2203.00261v3>
3. Mello, J. P., Jr. (2025, July 31). Quantum delivers really random numbers: How that boosts AppSec. ReversingLabs. <https://www.reversinglabs.com/blog/quantum-comes-to-appsec>
4. "What is a quantum random number generator (QRNG)? overview," Palo Alto Networks, <https://www.paloaltonetworks.com/cyberpedia/what-is-a-quantum-random-number-generator-qrng#where-are-quantum-random-number-generators-being-used-today> (accessed Apr. 29, 2026).
5. ANU QRNG – Quantum random numbers. (n.d.). <https://qrng.anu.edu.au/> (accessed May 4, 2026)

6. David Cirauqui, Miguel Ángel García-March, Guillem Guigó Corominas, Tobias Groß, Przemysław R. Grzybowski, Gorka Muñoz-Gil, J. R. M. Saavedra, Maciej Lewenstein; Comparing pseudo- and quantum-random number generators with Monte Carlo simulations. *APL Quantum* 1 September 2024; 1 (3): 036125. <https://doi.org/10.1063/5.0199568>
7. A. Rukhin *et al.*, "Special Publication 800-22 Revision 1a A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," Apr. 2010. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-22r1a.pdf>